

Universidad Técnica Estatal de Quevedo

Facultad de Ciencias de la Computación y Diseño Digital

Carrera de Ingeniería de Software



TA-IND-01 — Informe Académico Individual

Análisis Crítico de Sistemas Distribuidos

Unidad 1 — Netflix

Autor:

Jhinson Stalyn Aucatoma Celorio

Miércoles 13 de mayo del 2026

Repositorio GitHub

[Ver repositorio en GitHub](#)

Resumen En este informe se realiza un análisis crítico a Netflix como un sistema distribuido real, relacionándolo y comparándolo con los conceptos principales de la Unidad 1 de la asignatura Aplicaciones Distribuidas. El análisis se centra en las transparencias de distribución, el modelo CAP, los modelos de comunicación y los mecanismos de tolerancia a fallos utilizados por la plataforma. A partir de la revisión realizada, se evidencia que Netflix oculta gran parte de la complejidad de su infraestructura mediante servicios como Eureka, Open Connect, EVCache, Spinnaker, Kafka y gRPC. Sin embargo, estas transparencias no son absolutas, ya que algunas dependen del contexto técnico y pueden presentar limitaciones. Desde el modelo CAP, Netflix no puede clasificarse únicamente como AP o CP, debido a que prioriza disponibilidad en servicios orientados a la experiencia del usuario, pero exige consistencia en procesos críticos como pagos y facturación. Por tanto, Netflix es un ejemplo de que los sistemas distribuidos no eliminan los compromisos técnicos, sino que los administran de manera estratégica.

Palabras clave: sistemas distribuidos, Netflix, transparencia de distribución, modelo CAP, modelos de comunicación, tolerancia a fallos.

Abstract In this report, a critical analysis of Netflix as a real distributed system is carried out, relating and comparing it with the main concepts of Unit 1 of the Distributed Applications subject. The analysis focuses on the distribution transparencies, the CAP model, the communication models and the fault tolerance mechanisms used by the platform. From the review carried out, it is evident that Netflix hides much of the complexity of its infrastructure through services such as Eureka, Open Connect, EVCache, Spinnaker, Kafka and gRPC. However, these transparencies are not absolute, as some depend on the technical context and may present limitations. From the CAP model, Netflix cannot be classified solely as AP or CP, because it prioritizes availability in services oriented to the user experience, but requires consistency in critical processes such as payments and billing. Therefore, Netflix is an example that distributed systems do not eliminate technical commitments, but rather manage them strategically.

Índice

1. Introducción	3
2. Marco Teórico	3
2.1. Sistemas distribuidos	3
2.2. Transparencias de distribución	3
2.3. Modelo CAP	3
2.4. Modelos de comunicación	4
2.5. Tolerancia a fallos	4
3. Análisis Crítico de Netflix	4
3.1. Transparencia de distribución	4
3.2. Modelo CAP en Netflix	5
3.3. Modelos de comunicación en Netflix	6
3.4. Tolerancia a fallos	6
4. Conclusiones	7
Referencias	7

1. Introducción

Netflix es una de las plataformas digitales más representativas para analizar el funcionamiento de un sistema distribuido real, debido a que ofrece servicios de *streaming* a millones de usuarios desde diferentes lugares del mundo. Aunque para el usuario la experiencia parece simple —abrir la aplicación, elegir una película o serie y reproducirla—, detrás de esa acción intervienen múltiples componentes distribuidos: servidores, microservicios, sistemas de caché, mecanismos de comunicación, bases de datos y herramientas de tolerancia a fallos.

El presente informe tiene como finalidad analizar de manera crítica a Netflix aplicando los conceptos aprendidos en la Unidad 1 de la asignatura Aplicaciones Distribuidas. Para ello, se revisan las transparencias de distribución, el modelo CAP, los modelos de comunicación y los mecanismos de tolerancia a fallos presentes en su arquitectura.

2. Marco Teórico

2.1 *Sistemas distribuidos*

Un sistema distribuido es un conjunto de componentes de hardware y software interconectados mediante una red que coordinan sus acciones a través del paso de mensajes para lograr un objetivo común. Estos sistemas se caracterizan por tres propiedades esenciales: la concurrencia transparente, la ausencia de un reloj compartido y la necesidad de operar de manera fiable y tolerante a fallos [1].

2.2 *Transparencias de distribución*

La transparencia de distribución consiste en ocultar al usuario y a las aplicaciones la complejidad de que los procesos y recursos se encuentren físicamente distribuidos en diferentes máquinas. Van Steen y Tanenbaum plantean que esta transparencia puede lograrse mediante una capa intermedia o *middleware*, la cual ofrece interfaces similares aunque internamente existan diferencias de ubicación, comunicación, réplica o recuperación [2]. Entre las principales formas de transparencia se encuentran: acceso, ubicación, migración, reubicación, réplica, concurrencia y fallo.

2.3 *Modelo CAP*

El **Teorema CAP** explica que un sistema de datos distribuido no puede cumplir simultáneamente con tres propiedades:

- **Consistencia (C):** todos los nodos muestran la misma información.
- **Disponibilidad (A):** se garantiza una respuesta para cada consulta.
- **Tolerancia a particiones (P):** el sistema sigue funcionando si la red falla.

Como las redes en Internet siempre presentan fallas o congestión, los sistemas deben elegir entre consistencia o disponibilidad ante una partición de red [3]. Los sistemas **AP** (como Cassandra o DynamoDB) priorizan disponibilidad con consistencia eventual; los sistemas **CP** (como MongoDB o HBase) priorizan consistencia bloqueando consultas ante fallos; y los sistemas **CA** (como MySQL o PostgreSQL en redes locales) priorizan consistencia y disponibilidad asumiendo que la red no fallará.

2.4 Modelos de comunicación

La comunicación es un elemento central en los sistemas distribuidos porque los procesos necesitan intercambiar datos aunque se encuentren en máquinas diferentes. El modelo **cliente-servidor** es el más común en servicios web y APIs. También existen mecanismos como **RPC**, cuyo objetivo es permitir que un programa invoque procedimientos en otra máquina como si fueran llamadas locales; el cliente usa *stubs* que empaquetan parámetros, envían mensajes por la red y reciben resultados, ocultando parte de la comunicación distribuida [2].

2.5 Tolerancia a fallos

La tolerancia a fallos se refiere a la capacidad de un sistema distribuido para continuar funcionando cuando algunos de sus componentes fallan. A diferencia de un sistema centralizado, en un sistema distribuido pueden ocurrir fallos parciales: un nodo puede caerse, una red puede congestionarse, un servicio puede responder lentamente o una réplica puede quedar temporalmente inaccesible [2].

3. Análisis Crítico de Netflix

3.1 Transparencia de distribución

Van Steen y Tanenbaum [2] definen la transparencia como la capacidad de ocultar la complejidad de la distribución al usuario y al programador de aplicaciones. Identifican ocho tipos de transparencia que un sistema distribuido debe ofrecer: acceso, ubicación, migración, reubicación, réplica, concurrencia, fallo y persistencia.

La **transparencia de acceso** en Netflix es parcial porque usa varios mecanismos de comunicación: REST para ciertas APIs, gRPC para comunicación *backend-to-backend* y Kafka para eventos asíncronos [4], [5]. Esto demuestra que una transparencia puede ser sólida en un dominio, pero no necesariamente universal.

La **transparencia de ubicación** es una de las más fuertes. Eureka permite que los clientes localicen servicios sin depender de direcciones fijas, y Open Connect oculta al usuario desde qué *appliance* recibe los segmentos de video [6], [7].

La **transparencia de migración y reubicación** aparece en los despliegues con Spinnaker y Kayenta, que permiten liberar versiones de forma progresiva y revertir si el canary no es seguro [8]. Sin embargo, la reubicación es la más debatible porque mover tráfico activo entre regiones puede generar una ventana mínima de reconexión.

La **transparencia de réplica** se expresa en EVCache y Open Connect. EVCache oculta parte de la complejidad de la réplica de datos, pero no debe confundirse con almacenamiento persistente absoluto, ya que una caché puede perder datos por expulsión o reinicio [9].

Finalmente, la **transparencia de fallo** se apoya en *timeouts*, reintentos controlados, *circuit breakers* y pruebas de caos. Hystrix surgió para evitar que fallos de dependencias remotas provoquen una caída en cascada [10].

3.2 Modelo CAP en Netflix

El teorema CAP plantea que, frente a una partición de red, un sistema distribuido no puede garantizar simultáneamente consistencia fuerte, disponibilidad y tolerancia a particiones [11]. En Netflix, la tolerancia a particiones es inevitable porque sus componentes operan en múltiples zonas, regiones y redes.

Netflix suele **priorizar disponibilidad** en servicios donde una inconsistencia temporal no afecta de forma crítica al negocio: historial de visualización, recomendaciones o métricas. Este comportamiento se acerca al modelo AP con consistencia eventual, apoyado en Cassandra [12].

El pipeline de eventos basado en Kafka muestra un caso claro de priorización de disponibilidad. Netflix acepta una pérdida menor de mensajes para no bloquear a las aplicaciones productoras cuando un broker responde lentamente [5]. En eventos analíticos de gran volumen, bloquear la aplicación principal puede ser más dañino que perder una fracción pequeña de eventos.

En cambio, para **facturación y pagos**, Netflix adopta una postura más cercana a CP. Las transacciones financieras requieren ACID porque un error de consistencia puede produ-

cir cobros incorrectos o reclamos legales. Por eso se usa MySQL para operaciones de pago [13].

La conclusión crítica es que Netflix no debe clasificarse simplemente como “AP” o “CP”. Su arquitectura aplica CAP por contexto: AP para experiencia de usuario y datos tolerantes a inconsistencias temporales; CP para pagos y procesos donde la exactitud es prioritaria.

3.3 Modelos de comunicación en Netflix

Netflix usa HTTPS sobre TCP incluso en *streaming* en vivo. Aunque UDP puede ofrecer menor latencia, Netflix prioriza la compatibilidad con dispositivos y sistemas de entrega existentes [14]. Esta decisión se conecta con la falacia de Deutsch de que la red es homogénea: Netflix no puede asumir que todos los televisores, navegadores, consolas y teléfonos soporten de manera uniforme un protocolo específico.

En la comunicación interna, Netflix migró de HTTP/1.1 a gRPC para mejorar la definición, descubrimiento y uso de APIs entre microservicios [15]. Este cambio se relaciona con RMI/RPC: la idea base es invocar funcionalidad remota como si fuera una llamada local, pero aceptando que una llamada remota introduce latencia, consumo de red y probabilidad de error [4].

Kafka representa el modelo asíncrono Pub/Sub. En lugar de que todos los servicios esperen respuestas inmediatas, los productores publican eventos y los consumidores los procesan posteriormente, lo que desacopla componentes y mejora disponibilidad [5].

El monitoreo de conexiones TCP mediante FlowExporter y eBPF amplía el concepto de sockets a escala de millones de flujos para comprender estado de sockets, latencias, *timeouts* y cierres de conexión [16].

3.4 Tolerancia a fallos

La tolerancia a fallos en Netflix se basa en aceptar que las dependencias remotas fallan. Si una solicitud del API se expande hacia múltiples servicios, la probabilidad de que alguna dependencia responda lento o falle aumenta. Por ello, Netflix usa patrones como *timeouts*, *fallback*, *circuit breaker* y aislamiento de recursos [17].

Hystrix ejemplifica el patrón *circuit breaker*: cuando una dependencia falla repetidamente o supera el tiempo permitido, el circuito se abre para evitar que más solicitudes consuman recursos esperando una respuesta que probablemente no llegará [10]. Esto protege al sistema frente a fallos en cascada y permite ofrecer respuestas degradadas.

La ingeniería del caos complementa esta estrategia. Herramientas como Chaos Monkey o Chaos Kong provocan fallos controlados para comprobar si el sistema puede recuperarse sin esperar a que el fallo ocurra en producción [18].

El *load shedding* también es esencial: cuando la carga supera la capacidad, Netflix prioriza solicitudes críticas y rechaza o retrasa las menos importantes [19].

4. Conclusiones

El análisis realizado permite concluir que Netflix es un ejemplo claro de sistema distribuido moderno, ya que integra múltiples servicios, regiones, réplicas, protocolos de comunicación y mecanismos de recuperación para ofrecer una experiencia continua al usuario.

En relación con las transparencias de distribución, Netflix logra ocultar gran parte de la complejidad técnica al usuario final: la ubicación de los servidores, la existencia de réplicas, el uso de cachés y los procesos de recuperación no son visibles para quien reproduce contenido.

Respecto al modelo CAP, Netflix no puede clasificarse de forma simple como AP o CP. La plataforma prioriza disponibilidad en servicios relacionados con la experiencia del usuario, como reproducción, recomendaciones, historial y eventos analíticos. En cambio, para procesos críticos como pagos y facturación, la consistencia adquiere mayor importancia.

Adicionalmente, Netflix combina HTTPS sobre TCP, gRPC, Kafka y monitoreo de conexiones para responder a necesidades distintas, lo que demuestra que no existe un único modelo de comunicación “ideal”, sino que cada mecanismo se elige según el tipo de interacción, la latencia aceptable y el volumen de datos.

Finalmente, la tolerancia a fallos es uno de los aspectos más importantes de la arquitectura de Netflix. Herramientas como *circuit breakers*, *fallbacks*, pruebas de caos y *load shedding* permiten que el sistema no dependa de que todos sus componentes funcionen perfectamente al mismo tiempo.

Referencias

- [1] D. Lindsay, S. S. Gill, D. Smirnova y P. Garraghan, “The Evolution of Distributed Computing Systems: From Fundamentals to New Frontiers”, *Computing*, vol. 103, págs. 1859-1883, ene. de 2021. DOI: [10.1007/s00607-020-00900-y](https://doi.org/10.1007/s00607-020-00900-y).

- [2] M. van Steen y A. S. Tanenbaum, *Distributed Systems*, 4.^a ed. Maastricht, The Netherlands: Maarten van Steen, 2025, ver. 4.03. dirección: <https://www.distributed-systems.net>.
- [3] O. Drozd, S. Lazarenko, A. Maevsky, K. Drozd y A. Berdnikov, “The CAP Theorem in Distributed Systems”, en *Proc. IEEE 15th Int. Conf. DESSERT*, 2020, págs. 1-6. DOI: [10.1109/DESSERT50317.2020.9125078](https://doi.org/10.1109/DESSERT50317.2020.9125078).
- [4] Netflix Technology Blog, *Practical API Design at Netflix, Part 1: Using Protobuf FieldMask*, Medium, 2020. dirección: <https://netflixtechblog.com/practical-api-design-at-netflix-part-1-using-protobuf-fieldmask-35cfdc606518>.
- [5] Netflix Technology Blog, *Kafka inside Keystone Pipeline*, Medium, 2016. dirección: <https://netflixtechblog.com/kafka-inside-keystone-pipeline-dd5aeabaf6bb>.
- [6] Netflix Technology Blog, *Netflix Shares Cloud Load Balancing and Failover Tool: Eureka*, Medium, 2013. dirección: <https://netflixtechblog.com/netflix-shares-cloud-load-balancing-and-failover-tool-eureka-c10647ef95e5>.
- [7] Netflix Technology Blog, *Driving Content Delivery Efficiency through Classifying Cache Misses*, Medium, 2022. dirección: <https://netflixtechblog.com/driving-content-delivery-efficiency-through-classifying-cache-misses-ffcf08026b6c>.
- [8] Netflix Technology Blog, *Automated Canary Analysis at Netflix with Kayenta*, Medium, 2018. dirección: <https://netflixtechblog.com/automated-canary-analysis-at-netflix-with-kayenta-3260bc7acc69>.
- [9] Netflix Technology Blog, *Caching for a Global Netflix*, Medium, 2016. dirección: <https://netflixtechblog.com/caching-for-a-global-netflix-7bcc457012f1>.
- [10] Netflix Technology Blog, *Introducing Hystrix for Resilience Engineering*, Medium, 2012. dirección: <https://netflixtechblog.com/introducing-hystrix-for-resilience-engineering-13531c1ab362>.
- [11] E. Brewer, “Towards Robust Distributed Systems”, en *Proc. 20th ACM Symp. Principles of Distributed Computing (PODC)*, Portland, OR, USA, 2000. DOI: [10.1145/564585.564601](https://doi.org/10.1145/564585.564601).
- [12] Netflix Technology Blog, *Scaling Time Series Data Storage — Part I*, Medium, 2020. dirección: <https://netflixtechblog.com/scaling-time-series-data-storage-part-i-ec2b6d44ba39>.

- [13] Netflix Technology Blog, *Netflix Billing Migration to AWS*, Medium, 2016. dirección: <https://netflixtechblog.com/netflix-billing-migration-to-aws-451fba085a4>.
- [14] Netflix Technology Blog, *Behind the Streams: Live at Netflix Part 1*, Medium, 2023. dirección: <https://netflixtechblog.com/behind-the-streams-live-at-netflix-part-1-d23f917c2f40>.
- [15] Cloud Native Computing Foundation (CNCF), *Netflix Case Study*, CNCF, 2022. dirección: <https://www.cncf.io/case-studies/netflix/>.
- [16] Netflix Technology Blog, *How Netflix Accurately Attributes eBPF Flow Logs*, Medium, 2023. dirección: <https://netflixtechblog.com/how-netflix-accurately-attributes-ebpf-flow-logs-afe6d644a3bc>.
- [17] Netflix Technology Blog, *Fault Tolerance in a High Volume, Distributed System*, Medium, 2012. dirección: <https://netflixtechblog.com/fault-tolerance-in-a-high-volume-distributed-system-91ab4faae74a>.
- [18] Netflix Technology Blog, *The Netflix Simian Army*, Medium, 2011. dirección: <https://netflixtechblog.com/the-netflix-simian-army-16e57fbab116>.
- [19] Netflix Technology Blog, *Keeping Netflix Reliable Using Prioritized Load Shedding*, Medium, 2021. dirección: <https://netflixtechblog.com/keeping-netflix-reliable-using-prioritized-load-shedding-6cc827b02f94>.